

Classification of Periodic Genes in Yeast Gene Expression Data Using Machine Learning

Özlem AKINCI

Yalova University, Computer Engineering MSc Program, Yalova, Turkey

ABSTRACT

This study is a classification work in which the periodicity of yeast (*Saccharomyces cerevisiae*) genes, accepted as the gold standard, is identified through machine learning methods. The dataset to be used for analysis is spellman.csv (CDC15 time series), published in 1998 and known as the Spellman dataset. The 800-gene periodic reference list, which will serve as the dependent variable and will be used to examine periodicity, was retrieved using the R programming language and will function as the label.

Although the study was essentially planned as a 3-stage process, it consists of many sub-steps. With the completion of preprocessing and feature engineering steps, results were optimized in a staged manner. As the core framework, 13 different machine learning algorithms were tested in the first stage. Among these, LightGBM achieved the highest macro F1 score (0.8413) while CatBoost produced the highest AUC value (0.9199). Based on the results, the process returned to the feature extraction step to perform a more comprehensive feature extraction. Subsequently, five models demonstrating high F1 and AUC values with distinct learning mechanisms — Random Forest, Extra Trees, XGBoost, LightGBM, and CatBoost — among the initial 13 models were selected for use in the ensemble model following the expanded feature extraction. In the project, which progressed through three stages, the best results were achieved with the ensemble model, which attained a macro F1 score of 0.8168, MCC of 0.6378, and AUC of 0.9056.

Keywords: Gene Expression, Cell Cycle, Periodic Gene Detection, Machine Learning, Feature Engineering, SMOTE, Ensemble Learning

1. INTRODUCTION

The cell cycle is a complex biological process fundamental to the growth, development, and reproduction of organisms [2]. Understanding this process is largely embedded in gene expression data. The process relies on the detection of periodic behaviors in the data [3]. Spellman and colleagues, whose work on the yeast (*Saccharomyces cerevisiae*) cell cycle is considered a cornerstone of computational biology, demonstrated using microarray hybridization that approximately 800 genes are periodically regulated by the cell cycle [1]. The list of periodic genes identified through objective statistical tests in this pioneering study has been used as a "gold standard" or benchmark set [4,5].

The identification of periodic genes from microarray time series data is considered a highly challenging problem due to reasons such as the extremely noisy nature of the data, missing

observations, and short sample lengths [2,3,4]. To overcome these challenges and enable the detection of periodicity, various mathematical approaches have been proposed, including Fourier transforms [2,4], the Lomb-Scargle periodogram [6], Empirical Mode Decomposition (EMD) [7], and various robust spectral analysis algorithms. Nevertheless, the existence of serious inconsistencies between periodic gene lists identified by different algorithms [6] and the minority status of periodic genes give rise to the class imbalance problem [8]. For this reason, machine learning approaches capable of capturing nonlinear interactions and providing greater robustness are needed [9]. Examples of supervised machine learning algorithms such as Decision Tree Ensembles [10], Support Vector Machines (SVM), KNN, and RF [11] can be found in the literature. These methods stand out for their ability to capture nonlinear interactions and provide resistance to noise [12,13].

This study aims to present a comprehensive classification framework in which machine learning models are tested to determine whether yeast genes are periodic. Unlike conventional approaches, this research has designed a rich feature engineering phase for machine learning using autocorrelation features that indicate the relationship of the time series with its own past (particularly lag-1 and lag-2 delays) and sinusoidal curve fitting parameters (amplitude, frequency, and phase) representing the periodic characteristics of the gene. Since machine learning models are adversely affected by the class imbalance problem frequently encountered in biological data, the SMOTETomek hybrid technique (oversampling/undersampling) was employed in this study to accurately identify the minority periodic gene class. An ImbPipeline architecture was established to prevent information leakage from the training data to the test data. Using a decision tree ensemble model trained with cross-validation, the consistency of the algorithms in predicting positive and negative classes was evaluated through macro F1 metrics. The findings obtained reveal that the feature engineering and data balancing strategies applied offer an effective solution for overcoming the class imbalance and noise obstacles encountered in the discovery of periodic genes in gene expression data.

2. METHODS

This study used machine learning models to determine whether yeast (*Saccharomyces cerevisiae*) genes are periodic using classification methods.

2.1 Dataset aand Labeling

In the study, the "Spellman.csv" dataset published by Spellman et al. and the reference list "spellman_periodic_genes.txt" containing 800 periodic genes [1] were used. In this context, the dataset to be analyzed (spellman.csv) and the 800-gene periodic list (gold standard) were used as the dependent variable (label).

The Spellman CDC15 time series dataset (spellman.csv, the raw feature matrix used in the study) consists of time series gene expression (microarray) data obtained from yeast cells synchronized using a temperature-sensitive cdc15 mutant [1,6]. Spellman and colleagues used multiple synchronization methods (alpha factor, elutriation, and cdc15, cdc28 mutants) to identify genes periodically expressed in the yeast (*Saccharomyces cerevisiae*) cell cycle [1]. The 800-gene periodic list added externally to the project dataset was defined not by examining a single dataset (e.g., only CDC15), but by combining data from alpha, cdc15, and

cdc28 experiments, applying Fourier transform (periodicity test) and correlation algorithms with known genes (Pearson), and exceeding an objectively calculated "CDC score" threshold. The 800-gene periodic list in question does not exist directly as a label within the CDC15 raw dataset itself. This list was constructed through the mathematical synthesis of three different experiments [1]. It will function as the positive class (periodic / label=1) in the machine learning models of the project. The remaining transcripts in the dataset will be accepted as non-periodic (negative class / label=0) genes.

Samples were taken every 10 minutes for 290 minutes from the culture entering the cell cycle. However, due to the constraints of the biological experiment, observations at the 20th, 40th, 60th, 260th, and 280th minutes are missing (missing values), reducing the total number of analyzable time points to 24 [6,9].

In Spellman's original microarray experiment, approximately 6178 Open Reading Frames (ORFs) representing the yeast genome were measured [6]. The 4381-transcript list used in our study is a cleaned subset obtained by filtering out genes with too many missing measurements or those that were not expressed at all (noisy). In the literature, various algorithms prefer to fill in these missing values (using spline or Lomb-Scargle interpolation, for example) or to process the data as-is with missing values [6,14].

The use of the "800-gene" list as a positive class is also quite common in the literature. Zhao et al. (2009) used it to measure the success rate of their developed feature extraction methods and for comparison purposes (as True Positive / False Positive) [4]. Studies such as Wu and Li [14] and Cheng et al. [15] trained classifiers such as Logistic Regression (PLR) and Support Vector Machines (SVM) by setting Spellman's 800 genes as the "Positive Class" (label = 1) and other genes known to be non-periodic as the "Negative Class" (label = 0). After these models were trained, they were applied to the other (unlabeled) genes in the dataset, and brand new periodic genes (novel genes) not included in Spellman's 800 genes but labeled as "periodic" by artificial intelligence were discovered. In the literature, there are studies similar to ours in which the gold standard (800 genes as known positive/negative labels) is used to train machine learning models (particularly SVM and Decision Trees) to discover new periodic genes or relationships [10,12].

Our study is based on matching the 4381-gene (CDC15) independent variable matrix (X) with 24 time points and missing observations, with the consensus 800-gene dependent variable (y) label proven by Spellman's Fourier tests, in parallel with common machine learning applications in the literature.

2.2. Exploratory Data Analysis (EDA)

Exploratory data analysis is used to summarize the main features of the data, understand its structure, and reveal patterns within it, before the dataset is modeled or before moving to the testing phase. In the literature, the "average periodogram" method developed by Wichert and colleagues for the detection of possible periodic genes in gene expressions has been used directly as an exploratory tool in sources [6]. In addition, methods such as dimensionality

reduction (PCA), clustering [16], and Empirical Mode Decomposition (EMD) have also been used to understand and discover hidden structures in gene expressions [7].

In this project, exploratory data analysis was used to make basic observations on the features that determine whether genes are periodic.

The EDA phase consists of three analysis steps. First, the distribution of gene expression values at each time point according to classes was examined by creating histogram and kernel density estimation (KDE) plots. This analysis was used to visualize the extent of class imbalance and the degree of overlap between the two classes. In the second step, the Pearson correlation matrix was calculated and visualized as a heatmap. In the final step, box plots were created to compare the median, interquartile range, and outlier distribution of periodic and non-periodic genes at each time point.

Based on the results obtained from this step, it was decided to use SMOTE for data imbalance in subsequent stages, and feature engineering methods such as Fourier Transform and Autocorrelation for capturing information such as the amplitude, frequency, and phase of the time series.

2.3. Preprocessing: Missing Value Management and Normalization

At this stage of the project, first, the expression values of 4381 genes in the Spellman dataset at 23 time points were scanned and missing value (NaN) analysis was performed. Since the cleaned version of the dataset with missing values removed was used in the study, no missing values (NaN) were found in the dataset. Therefore, no imputation process was performed for the missing value management step. In the literature, preprocessing steps have been applied to the original Spellman datasets in various studies. For preprocessing in the literature, while Wu and Li preferred to exclude genes with more than one missing value in the time series profile [14], Zhang et al. preferred to completely discard genes with more than 30% missing values [2]. While Filkov and colleagues preferred to filter out meaninglessly narrow peaks or troughs [5], Chen and colleagues applied the Empirical Mode Decomposition (EMD) method [7].

Following feature extraction, StandardScaler was applied in the study for the normalization step. This method was preferred to prevent features at different scales from adversely affecting the performance of distance- and gradient-based algorithms. While the scaling process was applied only to the training set, the transformation was applied to both the training and test sets. This approach was applied only to SVM, KNN, MLP, and Logistic Regression. Decision tree-based models were trained on unscaled raw features.

2.4. Feature Engineering

Instead of feeding the raw 23-point time series data directly into the model, a feature engineering approach was used to extract features from the gene expression data (X_{raw}). This method was employed because genes can vary in length and scale. Feature engineering was developed in two stages:

a. Basic Features (Stage 1 - 13 features)

In the first stage, the following feature groups were calculated to form the basis for algorithm comparison.

Basic Statistical Features: For each gene's expression profile, 5 features were calculated: mean, standard deviation (std), maximum (max), minimum (min), and range. As a result of this process, each gene was transformed into a fixed-size and comparable vector.

Fourier Transform (FFT) Features: The expression profile of a periodic gene creates power accumulation at specific harmonics in the frequency domain. FFT quantifies periodicity much more clearly than observing it in the time domain. Band separation (low / medium / high) was applied to distinguish the cell cycle-specific frequency range (~5–11th harmonic) from other biological noise. In summary, the FFT applied to each gene's time series data was used to extract features in the frequency domain.

Autocorrelation Features: Autocorrelation values were calculated for each gene's expression profile, particularly for lag-1 and lag-2 delays. This highlights periodicity by showing how correlated the time series is with its own past values.

While FFT captures the global frequency structure, autocorrelation measures local temporal dependency; when used together, they complement each other.

Peak Count: If a gene is truly periodic, it is expected to show regular rises and falls ('up-down' patterns) characterized by distinct minima and maxima throughout its expression profile [17,18,19]. The number of peaks during the expression process reduces this oscillation pattern to a single number. This feature is used as a fundamental criterion for distinguishing irregular or monotonic profiles from periodic candidates (filtering/penalization) [1].

b. Advanced Features:

In addition to the basic features in Stage 1, the following advanced features were used to better capture periodicity.

Sinusoidal Curve Fitting: In the literature, genes expressed in the cell cycle have been defined as genes showing sinusoidal expression oscillation with distinct minima and maxima [17]. In the study, a sine function ($A \times \sin(2\pi \times f \times t + \phi) + \text{offset}$) was fitted to gene expression profiles to extract the amplitude (A), frequency (f), phase (ϕ), and offset parameters representing the periodic characteristics of the gene. Neutral default values were assigned to genes for which the fitting process failed. These four parameters were added to the 13 features from Stage 1, raising the total number of features to 17.

Advanced feature extraction is performed immediately after Stage 1 is completed, before Stage 2 begins. All subsequent processes work with this 17-feature data.

2.5. Handling Class Imbalance (Oversampling)

As a result of the experiments conducted in the study, it was determined that the positive class ratio in the dataset was at the level of 14.4% (800 periodic genes - 14.4% and 3581 non-periodic genes - 85.6%). In the literature, it is emphasized that this imbalance leads to standard accuracy metrics concealing the true error rate; Ruan et al. gave an example of a

dataset with 990 negative and 10 positive samples where the model could achieve a misleadingly high accuracy of 99% by predicting "negative" for every instance [8].

In a study based on decision tree ensembles, the skewed ratio between classes was addressed by processing the majority class less and the minority class more during model training; the same study used metrics such as the Kappa statistic (K), which is much more reliable against data imbalance in performance measurements, to avoid the misleading nature of standard accuracy [8].

In our study; `class_weight='balanced_subsample'` was set for Random Forest and Extra Trees, `class_weight='balanced'` for other decision tree-based models, and the `scale_pos_weight` parameter was adjusted according to the class ratio in the training set for XGBoost and LightGBM. By activating such parameters, the focus was on addressing class imbalance at the algorithm level. In addition to this method, at the data level in Stage 2, SMOTE (Synthetic Minority Over-sampling Technique) was first applied to the training data to synthetically augment and balance the positive class for more accurate detection of periodic genes. In Stage 3, the most advanced Ensemble model employed the SMOTETomek oversampling and undersampling hybrid technique. This approach aimed to improve the learning capacity of the classifier by simultaneously cleaning noisy and overlapping majority samples through Tomek links while generating synthetic samples with SMOTE. The balancing process was applied only to the training set to prevent data leakage from the test set.

2.6. Model Selection and Training

Taking class imbalance into consideration, a three-stage model development strategy was followed. In all stages, the dataset was split once using stratified sampling into 75% training and 25% test sets, with class ratios preserved in both sets.

a. Stage 1 — Basic Model Comparison (13 features)

First, the following 13 models were trained using 13 basic features and their performances were compared:

- Random Forest
- Extra Trees
- Gradient Boosting
- XGBoost
- AdaBoost
- Decision Tree
- Naive Bayes
- Neural Network (MLP)
- RBF-Support Vector Machine (SVM)
- Logistic Regression
- K-Nearest Neighbors (KNN)
- LightGBM

- CatBoost

Distance and gradient based algorithms; SVM, KNN, MLP, and Logistic Regression were trained on data scaled with StandardScaler. Decision tree-based models were run on raw features. The purpose of this stage is to determine which algorithms show the best performance and to identify the models to be included in the Stage 3 ensemble model.

b. Stage 2 — Improved Random Forest (17 features)

Following Stage 1, the feature set was expanded to 17 by adding sinusoidal curve fitting parameters. With the expanded feature set, the Random Forest model was retrained on training data balanced with SMOTE. Hyperparameters were set as `n_estimators=500`, `max_depth=12`, `min_samples_leaf=2`, and `class_weight='balanced_subsample'`. This stage was designed to measure both the contribution of sinusoidal features and the improvement potential of a single model. Additionally, the generalizability of the model was evaluated using 5-fold StratifiedKfold cross-validation.

c. Stage 3 — Ensemble Model (17 features)

To achieve the best performance, five models that demonstrated high F1 and AUC values in Stage 1, with distinct learning mechanisms, were combined within a VotingClassifier framework.

Random Forest (`n_estimators=600`, `max_depth=14`), Extra Trees (`n_estimators=400`, `max_depth=14`), XGBoost (`n_estimators=400`, `learning_rate=0.08`, `max_depth=8`), LightGBM (`n_estimators=400`, `learning_rate=0.08`), CatBoost (`iterations=400`, `learning_rate=0.08`, `depth=8`).

The prediction probabilities of each model were combined using an unweighted average (soft voting). Prior to training, the SMOTETomek oversampling/undersampling technique was applied to the training set through an ImbPipeline. This ensures that the oversampling process is performed separately for each fold and prevents data leakage from the test set [8].

The ensemble model was trained on the training set of each fold and evaluated on the test set. The `f1_macro` (macro F1 score) metric was used to ensure balanced evaluation of both positive and negative classes. The mean and standard deviation of the F1 scores obtained from cross-validation were reported.

The final model was saved to the file `spellman_periodic_classifier_ensemble.pkl` for future use.

2.7. Evaluation Metrics and Threshold Optimization

Model performance was evaluated using standard classification metrics such as accuracy, precision, recall, and f1-score. Since the correct detection of periodic genes (capture rate) is critical, the focus was placed on the recall metric.

To optimize recall and f1-score values from the ensemble model's prediction probabilities, threshold values between 0.30 and 0.45 were scanned. The best threshold value was determined as the point that maintains a high recall value while providing a reasonable f1-score.

The final model was saved to the file `spellman_periodic_classifier_ensemble.pkl` for future use.

2.8. Model Validation and Generalizability Analysis

Model performance was evaluated using accuracy, macro precision, macro recall, macro F1 score, Cohen's Kappa coefficient, Matthews Correlation Coefficient (MCC), and AUC-ROC metrics.

Since the standard accuracy metric can be misleading under class imbalance conditions, MCC and Kappa coefficient, which are particularly robust against imbalance, were used as the primary comparison criteria. Due to the high biological cost of missing periodic genes (false negatives), the positive class recall value was additionally considered and monitored as an important criterion. In Stages 2 and 3, the classification threshold was lowered from the default value of 0.50 to increase the positive class recall value. For this purpose, threshold scanning was performed over the ranges of 0.30–0.45 (Stage 2) and 0.30–0.50 (Stage 3) based on prediction probabilities. The optimal threshold value was determined by balancing recall and macro F1.

3. RESULTS

In this section, the results of the models tested in line with the structure of the methods section will be discussed. This section will also present the results of the three-stage model development process established in the methods section.

3.1 Exploratory Data Analysis

Following the first operation in the methods section — merging the datasets — the negative class (0) count was 3748 while the positive class count was 633. A new and processable dataset named 'Spellman_with_labels.csv' with a total of 24 columns and 4381 rows was obtained. Three methods were used in this section.

a. Distribution Analysis

In the resulting graphs (figure 1), it can be seen that the vast majority of the data shows a tendency close to normal distribution. The blue areas (Non-Periodic - 0) on the graphs are much more dominant than the green areas (Periodic - 1). This indicates that the number of non-periodic genes in the dataset is much greater than the periodic ones, and that there is a class imbalance problem.

At almost all time points (40 - 210), the distributions of genes overlap to a large extent. This means that it is difficult to distinguish periodicity by looking at the value at a single time point.

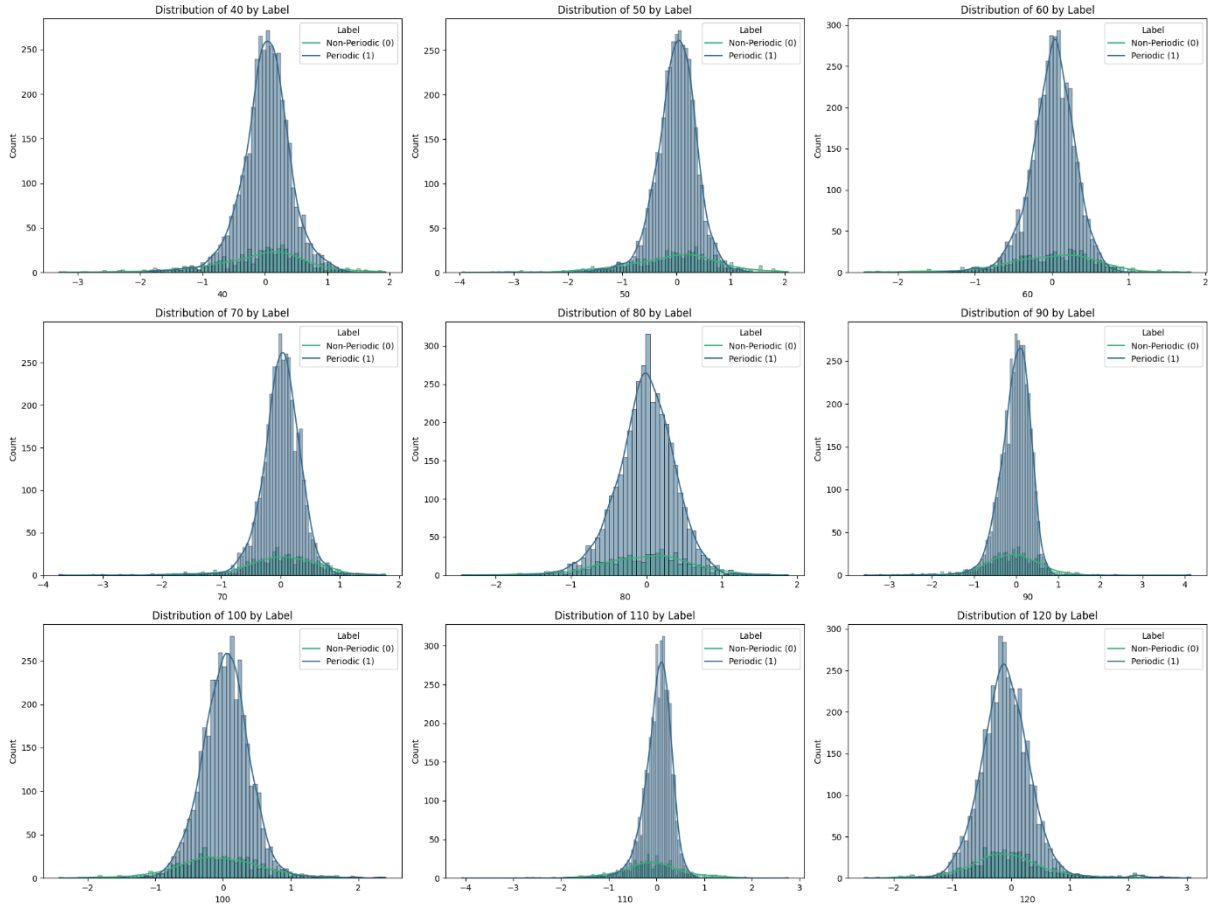


Figure 1: Resulting graphs of last dataset version

b. Outlier Analysis (Boxplot)

In figure 2, the median values of the periodic (1) and non-periodic (0) classes are quite close to each other. The box widths and whisker lengths of periodic genes (green boxes) are generally greater. This confirms that the expression values of periodic genes fluctuate over a wider range over time. In both groups, there are many outliers, especially in the non-periodic group. This indicates that there may be noise in the data.

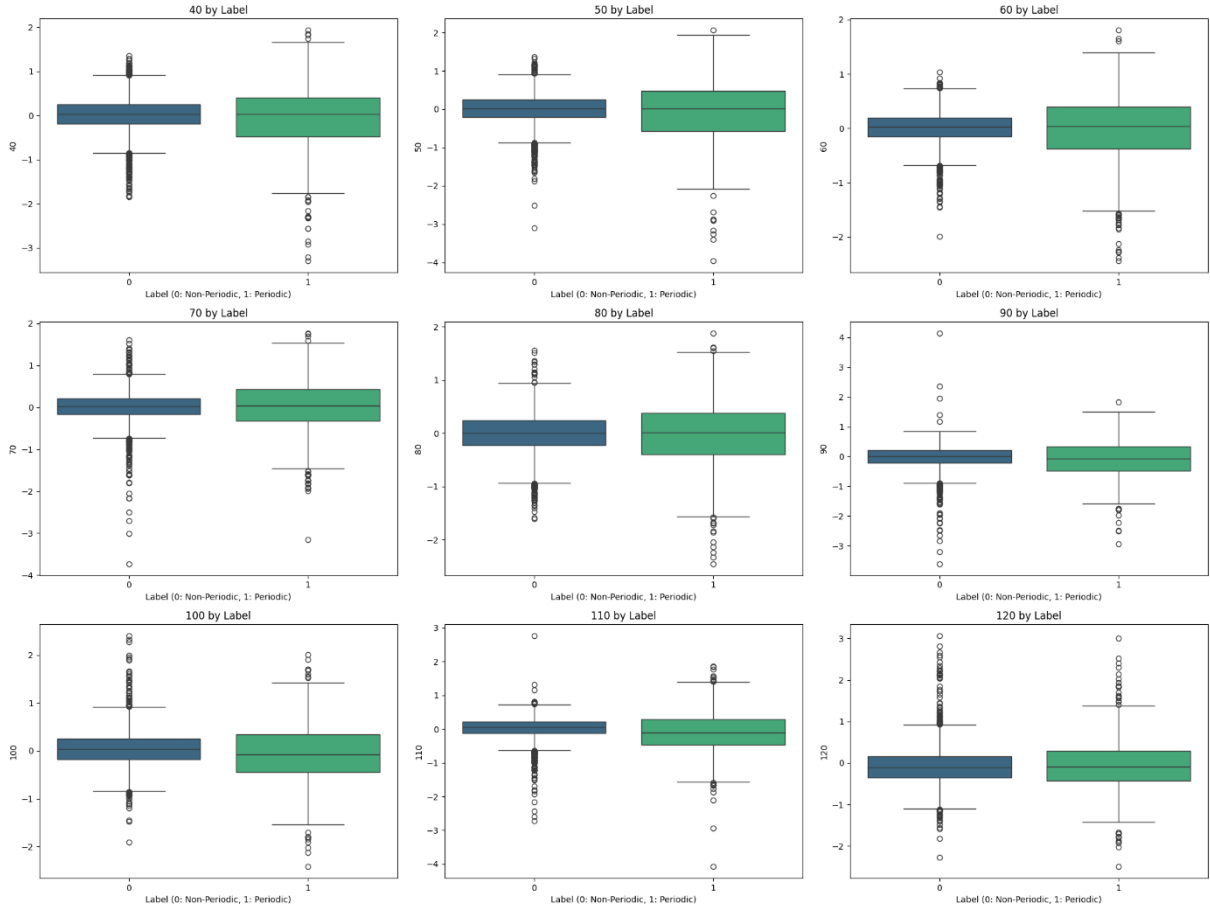


Figure 2: Outlier Analysis

c. Correlation Matrix

Very high positive correlation in figure 3 is observed between time points close to each other (0.91 between time points 40 and 50). The decrease in correlation with increasing time intervals indicates temporal continuity in expressions.

The correlation values and upward trend between the "label" column and time points are quite low (-0.07 - 0.16). From this result, we can see that whether a gene is periodic or not does not have a linear relationship based solely on its value at a specific time point. This result also leads to the inference that linear models will not perform well in the model training phase. The path to follow should be based on trying non-linear models and the interpretation that better results can likely be obtained.

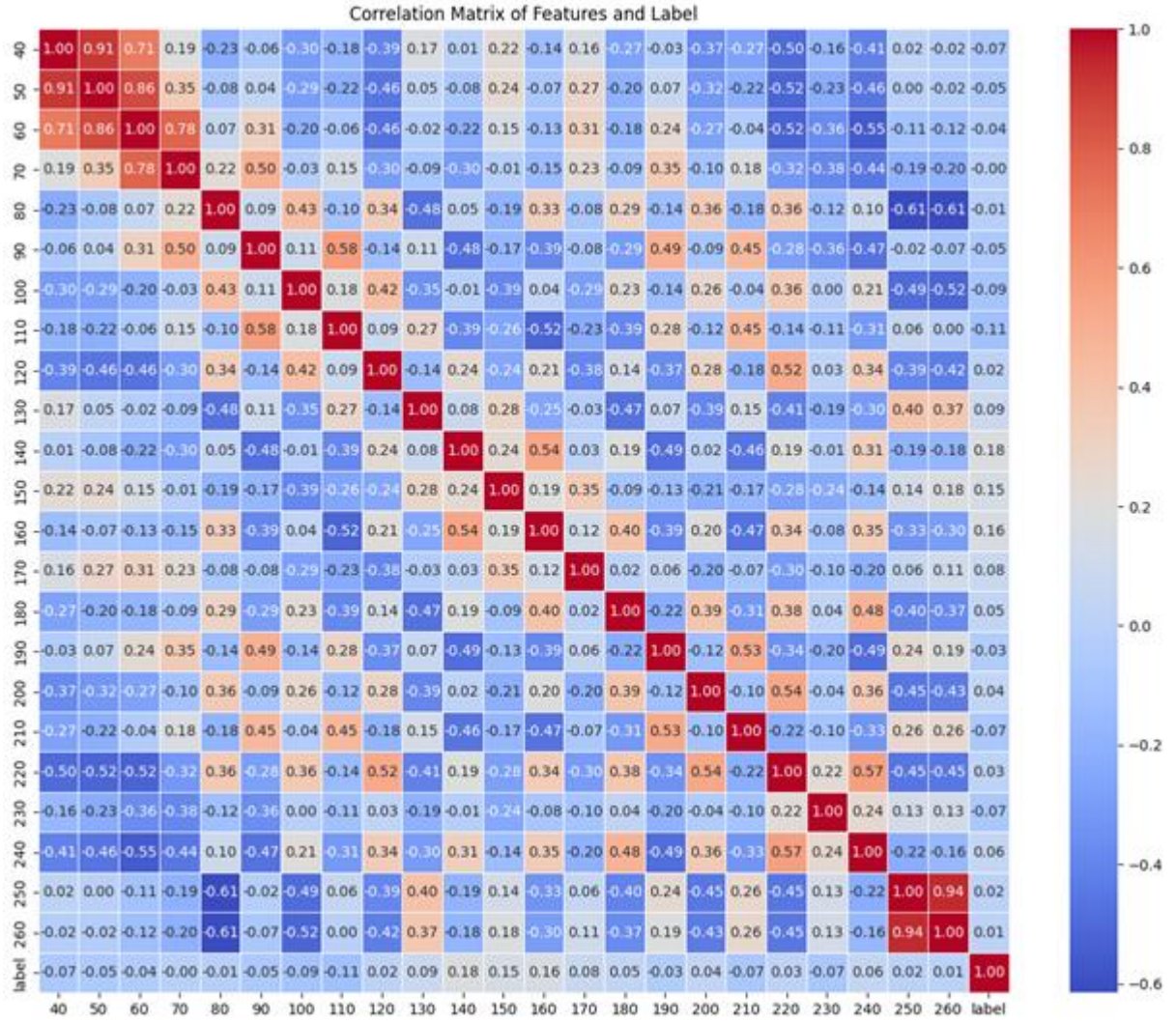


Figure 3: Correlation Matrix of dataset

All steps for the detection of periodicity in the study were designed as sequential steps following one another within a single flow. Dataset construction and preprocessing steps were handled as the foundation, while the main body was formed through the combination of feature engineering and model training phases. The structure referred to as the main body is divided into 3 stages in the methods section and will likewise be examined as 3 stages in the results section.

3.2 Stage 1: Basic Model Comparison

Following feature extraction, 13 algorithms were tested with 13 features and the results are shown in Table 1.

Accuracy values ranged from 0.8476 (Naive Bayes) to 0.9279 (Random Forest); macro F1 scores ranged from 0.7072 to 0.8413. The highest macro F1 value was achieved by LightGBM (0.8413), while the highest AUC value belonged to CatBoost (0.9199).

Decision tree-based models generally produced higher MCC and AUC values compared to linear and distance-based methods. Naive Bayes showed the lowest performance (F1=0.7072, MCC=0.4160).

MODEL	Accuracy	Precision (M)	Recall (M)	F1 (M)	Kappa	MCC	AUC
LightGBM	0.9234	0.8490	0.8342	0.8413	0.6827	0.6830	0.9153
Gradient Boosting	0.9243	0.8646	0.8110	0.8346	0.6696	0.6735	0.9118
Random Forest	0.9279	0.8988	0.7895	0.8315	0.6646	0.6795	0.9102
Extra Trees	0.9261	0.8909	0.7884	0.8284	0.6582	0.6716	0.9074
AdaBoost	0.9197	0.8486	0.8110	0.8281	0.6565	0.6585	0.9052
MLP	0.9151	0.8293	0.8241	0.8267	0.6534	0.6534	0.9100
XGBoost	0.9151	0.8302	0.8215	0.8258	0.6515	0.6517	0.9119
CatBoost	0.8978	0.7879	0.8535	0.8147	0.6306	0.6380	0.9199
KNeighbors	0.9033	0.8214	0.7540	0.7817	0.5645	0.5715	0.8388
Decision Tree	0.8869	0.7696	0.7918	0.7799	0.5600	0.5609	0.7918
RBF SVM	0.8723	0.7492	0.8306	0.7790	0.5609	0.5741	0.8980
Logistic Reg.	0.8595	0.7357	0.8363	0.7681	0.5416	0.5631	0.8977
Naive Bayes	0.8476	0.6976	0.7189	0.7072	0.4148	0.4160	0.8425

Table 1: Results of 13 ML algorithms models

3.3 Stage 2: Improved Random Forest

The feature set was expanded to 17 by adding sinusoidal curve fitting parameters, and the training set was balanced by applying SMOTE.

- Before SMOTE → Negative: 2810 | Positive: 475
- After SMOTE → Negative: 2810 | Positive: 2810

In the study, instead of taking the default threshold as 0.50, experiments were conducted with the aim of using the values that show the best performance. Since the threshold comes into play in the decision of class membership, the study aimed to find the threshold that provides the "least harm" and therefore shows the best performance.

Threshold scanning results are summarized in Table 2. When the default threshold was lowered from 0.50 to 0.46, the positive class recall value increased from 0.7595 to 0.7722; at the same threshold, the macro F1 score was obtained as 0.8085.

Threshold	Recall (+)	Precision (+)	F1	MCC
0.310000	0.8291	0.4441	0.7343	0.5182
0.350000	0.8228	0.4943	0.7633	0.5601
0.400000	0.7975	0.5164	0.7715	0.5671
0.430000	0.7848	0.5611	0.7910	0.5965
0.450000	0.7785	0.5885	0.8020	0.6141
0.460000	0.7722	0.6070	0.8085	0.6244

Table 2: Results of treshold trials

Following the selection of the optimal threshold value of 0.46, the model trial results are given in Table 3.

	precision	recall	f1-score	support
Negatif	0.96	0.92	0.94	938
Pozitif	0.61	0.77	0.68	158
accuracy	-	-	0.90	1096
macro avg	0.78	0.84	0.81	1096
weighted avg	0.91	0.90	0.90	1096

Table 3: Optimal threshold results

Although the optimal threshold of 0.46 was found, when the stage trials were compared, better results were obtained in Stage 2 with the default threshold of 0.50, despite the very small difference from the 0.46 threshold (Table 4). In line with this result, it was decided not to keep the threshold value at 0.46 and to conduct another threshold trial in the next stage as well.

Model	Accuracy	Precision (M)	Recall (M)	F1(M)	Kappa	MCC	AUC
RF — Aşama 1 (13 özellik, eşik=0.50)	0.9024	0.7975	0.8324	0.8133	0.6269	0.6290	0.9014
RF — Aşama 2 (17 özellik, eşik=0.50)	0.9024	0.7975	0.8324	0.8133	0.6269	0.6290	0.9014

Model	Accuracy	Precision (M)	Recall (M)	F1(M)	Kappa	MCC	AUC
RF — Aşama 1 (13 özellik, eşik=0.50)	0.9024	0.7975	0.8324	0.8133	0.6269	0.6290	0.9014
RF — Aşama 2 (17 özellik, eşik=0.46)	0.8951	0.7834	0.8440	0.8085	0.6180	0.6244	0.9014

Table 4: Comparison results of the results obtained from the first two stages

3.4 Stage 3: Ensemble Model

The threshold scanning results for the ensemble model (Table 5). The F1(M) value increased monotonically as the threshold increased; at the 0.50 threshold, the highest macro F1=0.8168, MCC=0.6378, and AUC=0.9056 were obtained. At the optimal threshold=0.50, positive class recall=0.7595, precision=0.6349, and F1=0.68 values were reached.

Threshold	Recall(+)	Precision(+)	F1(M)	MCC	AUC
0.350000	0.7975	0.5502	0.7882	0.5941	0.9056
0.400000	0.7848	0.5741	0.7969	0.6064	0.9056
0.450000	0.7722	0.5980	0.8048	0.6179	0.9056
0.490000	0.7658	0.6237	0.8139	0.6332	0.9056
0.500000	0.7595	0.6349	0.8168	0.6378	0.9056

Table 5: Ensemble models threshold trials results

3.5 Inter-Stage Comparison

Stage 1 → Stage 2 → Stage 3 Comparison, results_store output

The three results obtained are as shown in Table 6 and table 7.

Model	Accuracy	Precision (M)	Recall (M)	F1 (M)	Kappa	MCC	AUC
Aşama 1 — RF (13 özellik, eşik=0.50)	0.9279	0.8988	0.7895	0.8315	0.6646	0.6795	0.9102
Aşama 2 — RF (17 özellik, eşik=0.46)	0.8951	0.7834	0.8440	0.8085	0.6180	0.6244	0.9014
Aşama 3 — Ensemble (17 özellik, eşik=0.50)	0.9024	0.7965	0.8430	0.8168	0.6342	0.6378	0.9056

Table 6: Comparison of performance criteria for the three stages

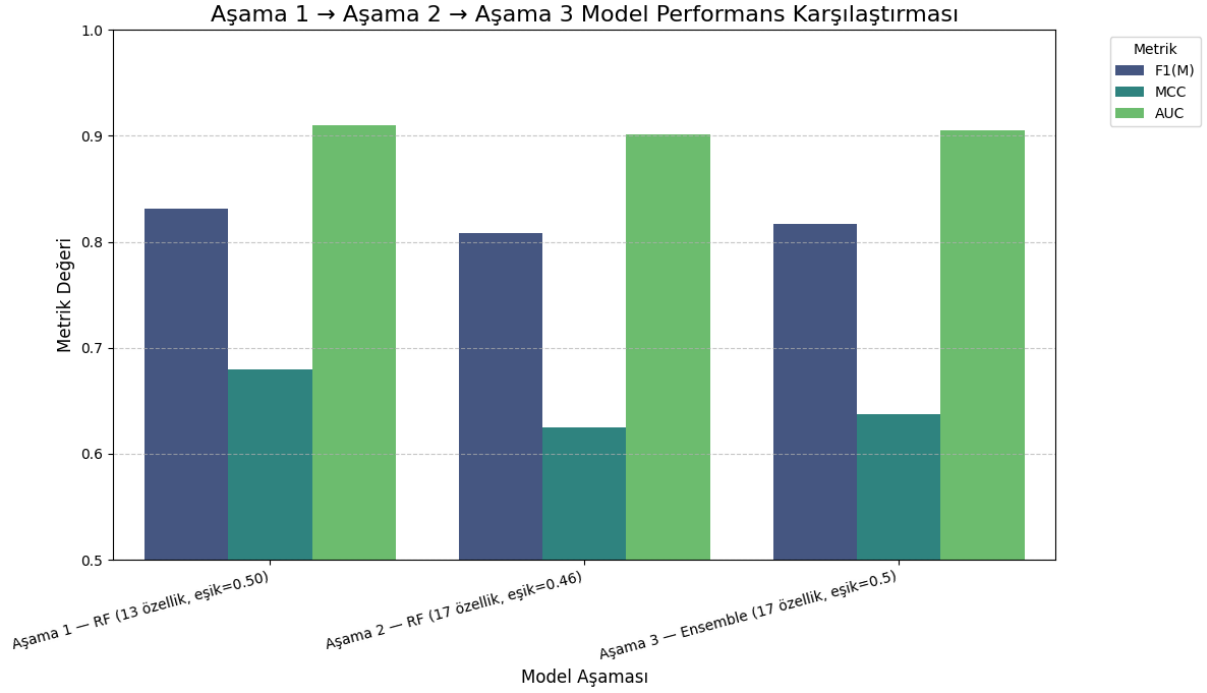


Table 7: Comparison of performance criteria for the three stages - column chart version

4. DISCUSSION

In this study, a three-stage machine learning framework was built to find out whether yeast genes show periodic expression. The results from each stage show, pretty clearly, what feature engineering and balancing strategies actually do to model performance.

Stage 1 tested 13 algorithms on 13 basic features. Decision tree-based models came out on top. Linear and distance-based methods fell behind. This lines up with what the literature says about ensemble methods handling nonlinear gene expression patterns better [12,13]. That said, Recall(M) values stayed around 0.79 which means the basic feature set was not quite enough to catch periodic genes reliably.

Stage 2 added sinusoidal curve fitting parameters. The feature count went from 13 to 17, and Recall(M) increase from 0.7895 to 0.8440. At 0.46, recall and F1 hit a reasonable balance. Cross-validation backed this up: macro F1 averaged 0.8526 ± 0.0140 across five folds, which is stable enough to trust.

In stage 3 five models, each with a different learning mechanism, were brought together under a VotingClassifier. SMOTETomek was used instead of SMOTE. SMOTETomek does not just add synthetic minority samples; it also cleans up the noisy, overlapping ones near the decision boundary. That cleanup showed up in the MCC and Kappa scores. Final numbers: F1(M)=0.8168, MCC=0.6378, AUC=0.9056.

A few limitations are mentioning. The data comes only from the CDC15 synchronization experiment. Bringing in alpha factor or cdc28 data could make the model more broadly applicable. Threshold selection here was driven purely by F1 and recall scores. Tying it to an actual biological cost function — how bad is a missed periodic gene versus a false alarm — would make future versions of this work considerably stronger.

REFERENCES

- [1] Spellman, P. T., Sherlock, G., Zhang, M. Q., Iyer, V. R., Anders, K., Eisen, M. B., ... & Futcher, B. (1998). Comprehensive identification of cell cycle-regulated genes of the yeast *Saccharomyces cerevisiae* by microarray hybridization. *Molecular biology of the cell*, 9(12), 3273-3297.
- [2] Zhao, W., Agyepong, K., Serpedin, E., & Dougherty, E. R. (2008). Detecting periodic genes from irregularly sampled gene expressions: a comparison study. *EURASIP Journal on Bioinformatics and Systems Biology*, 2008(1), 769293.
- [3] Agyepong, K. S., Hsu, F. H., Dougherty, E. R., & Serpedin, E. (2013). Spectral Analysis on Time-Course Expression Data: Detecting Periodic Genes Using a Real-Valued Iterative Adaptive Approach. *Advances in Bioinformatics*, 2013(1), 171530.
- [4] Zhao, W., Serpedin, E., & Dougherty, E. R. (2009). Identifying genes involved in cyclic processes by combining gene expression analysis and prior knowledge. *EURASIP Journal on Bioinformatics and Systems Biology*, 2009(1), 683463.
- [5] Filkov, V., Skiena, S., & Zhi, J. (2001, April). Analysis techniques for microarray time-series data. In *Proceedings of the fifth annual international conference on Computational biology* (pp. 124-131).
- [6] Liew, A. W. C., Xian, J., Wu, S., Smith, D., & Yan, H. (2007). Spectral estimation in unevenly sampled space of periodically expressed microarray time series data. *BMC bioinformatics*, 8(1), 137.
- [7] Chen, C. R., Shu, W. Y., Chang, C. W., & Hsu, I. C. (2014). Identification of under-detected periodicity in time-series microarray data by using empirical mode decomposition. *PloS one*, 9(11), e111719.
- [8] Ruan, J., Deng, Y., Perkins, E. J., & Zhang, W. (2009). An ensemble learning approach to reverse-engineering transcriptional regulatory networks from time-series gene expression data. *BMC genomics*, 10(Suppl 1), S8.
- [9] Andersson, C. R., Hvidsten, T. R., Isaksson, A., Gustafsson, M. G., & Komorowski, J. (2007). Revealing cell cycle control by combining model-based detection of periodic expression with novel cis-regulatory descriptors. *BMC Systems Biology*, 1(1), 45.
- [10] Huynh-Thu, V. A., Irrthum, A., Wehenkel, L., & Geurts, P. (2010). Inferring regulatory networks from expression data using tree-based methods. *PloS one*, 5(9), e12776.
- [11] Huang, F., Chen, L., Guo, W., Huang, T., & Cai, Y. D. (2022). Identification of Human Cell Cycle Phase Markers Based on Single-Cell RNA-Seq Data by Using Machine Learning Methods. *BioMed Research International*, 2022(1), 2516653.

- [12] Hegde, A., Nguyen, T., & Cheng, J. (2025). Machine learning methods for gene regulatory network inference. *Briefings in Bioinformatics*, 26(5), bbaf470.
- [13] Hart, C. E., Mjolsness, E., & Wold, B. J. (2006). Connectivity in the yeast cell cycle transcription network: inferences from neural networks. *PLoS Computational Biology*, 2(12), e169.
- [14] Wu, W. S., & Li, W. H. (2008). Systematic identification of yeast cell cycle transcription factors using multiple data sources. *BMC bioinformatics*, 9(1), 522.
- [15] Cheng, C., Fu, Y., Shen, L., & Gerstein, M. (2013). Identification of yeast cell cycle regulated genes based on genomic features. *BMC systems biology*, 7(1), 70.
- [16] Kondrakhin, Y. V., Podkolodnaya, O. A., Kochetov, A. V., Erokhin, G. N., & Kolchanov, N. A. (2004). Statistical Analysis of Microarray Data: Identification and Classification of Yeast Cell Cycle Genes. In *Bioinformatics of Genome Regulation and Structure* (pp. 331-341). Boston, MA: Springer US.
- [17] Panchy, N. L., Lloyd, J. P., & Shiu, S. H. (2020). Improved recovery of cell-cycle gene expression in *Saccharomyces cerevisiae* from regulatory interactions in multiple omics data. *BMC genomics*, 21(1), 159.
- [18] Cummins, B., Motta, F. C., Moseley, R. C., Deckard, A., Campione, S., Gedeon, T., ... & Haase, S. B. (2022). Experimental Guidance for Discovering Genetic Networks through Iterative Hypothesis Reduction on Time Series. *bioRxiv*, 2022-04.
- [19] Kelliher, C. M., Foster, M. W., Motta, F. C., Deckard, A., Soderblom, E. J., Moseley, M. A., & Haase, S. B. (2018). Layers of regulation of cell-cycle gene expression in the budding yeast *Saccharomyces cerevisiae*. *Molecular biology of the cell*, 29(22), 2644-2655